作业 4：Deep Ritz 方法求解 Poisson 方程

使用 CuPy 环境

2026 年 5 月 15 日

摘要

本报告实现了 Deep Ritz 方法来求解 Poisson 方程。Deep Ritz 是一种基于变分原理的无网格神经网络方法，将偏微分方程求解转化为神经网络参数优化问题。我们使用带残差连接（ResNet）的深层神经网络来逼近 Poisson 方程的解，通过最小化能量泛函（含边界罚项）来实现参数优化。在单位正方形区域上进行了数值实验，验证了方法的有效性，并与不含 ResNet 的网络结构进行了对比分析。实验结果表明，采用 ResNet 结构和随机配点采样的 Deep Ritz 方法能够有效求解 Poisson 方程。

一、问题描述

1.1 Poisson 方程与变分问题

本报告考虑以下 Dirichlet 边界条件的 Poisson 方程：

$$\begin{cases} -\Delta u(x) = f(x), & x \in \Omega, \\ u(x) = g(x), & x \in \partial\Omega. \end{cases} \tag{1}$$

其中 $\Omega \subset \mathbb{R}^2$ 是求解域，$f(x)$ 是源项函数，$g(x)$ 是边界值，Δ 为 Laplace 算子。

二、变分法理论与 Deep Ritz 原理

2.2 变分法证明定理

根据变分法，Poisson 方程的求解等价于最小化以下能量泛函：

$$J[u] = \int_\Omega \left(\frac{1}{2} |\nabla u(x)|^2 - f(x)u(x) \right) dx. \tag{2}$$

定理：在合适的函数空间上，若 $u^* = \arg\min_{u \in H} J[u]$，其中 $H = \{u \mid u(x) = g(x), x \in \partial\Omega\}$，则 u^* 满足 Poisson 方程。

证明（变分法推导）：

设 u^* 是能量泛函的最小值点。对任意满足 $v(x) = 0$ 在 $x \in \partial\Omega$ 的扰动 $v(x)$，考虑

$$J[u^* + \epsilon v] = \int_\Omega \left(\frac{1}{2} |\nabla u^* + \epsilon \nabla v|^2 - f(u^* + \epsilon v) \right) dx. \tag{3}$$

展开到一阶：

$$J[u^* + \epsilon v] = J[u^*] + \epsilon \int_\Omega (\nabla u^* \cdot \nabla v - fv) dx + O(\epsilon^2). \tag{4}$$

u^* 是最小值点的必要条件是一阶变分为零，即对所有 v 有

$$\delta J = \int_\Omega (\nabla u^* \cdot \nabla v - fv) dx = 0. \tag{5}$$

使用分部积分（Green 公式）：

$$\int_\Omega \nabla u^* \cdot \nabla v \, dx = -\int_\Omega (\Delta u^*) v \, dx + \int_{\partial\Omega} \frac{\partial u^*}{\partial n} v \, dS. \tag{6}$$

由于 $v = 0$ 在边界上，第二项消失：

$$\int_\Omega (-\Delta u^* - f) v \, dx = 0. \tag{7}$$

由于 v 为任意的（满足齐次边界条件），根据泛函分析的基本引理，必有

$$-\Delta u^* = f \text{ 在} \Omega \text{ 上}. \tag{8}$$

即 u^* 满足 Poisson 方程。 □

2.3 Deep Ritz 方法原理

Deep Ritz 方法的核心思想：使用参数化的神经网络 $u_\theta(x)$ 逼近精确解，将 PDE 求解转化为参数优化问题。

根据通用逼近定理，足够深和宽的神经网络可以以任意精度逼近任意连续函数。因此我们用网络输出 $u_\theta(x)$ 代替精确解 $u(x)$。

结合边界条件，实际求解时采用包含边界罚项的能量泛函：

$$L(\theta) = \int_\Omega \left(\frac{1}{2}|\nabla u_\theta(x)|^2 - f(x)u_\theta(x)\right) dx + \beta \int_{\partial\Omega} (u_\theta(x) - g(x))^2 dS. \tag{9}$$

其中 $\beta > 0$ 为罚参数。通过数值积分近似这些积分，使用 SGD 最小化损失函数 $L(\theta)$，最终得到数值解。

三、损失函数与数值求解

3.4 损失函数的具体形式

在实现中，使用 Monte Carlo 数值积分近似能量泛函：

1. **内部积分近似**：在域 Ω 内均匀随机采样 N_{int} 个点 $\{x_i\}_{i=1}^{N_{\text{int}}}$

$$\int_\Omega \left(\frac{1}{2}|\nabla u_\theta|^2 - f u_\theta\right) dx \approx |\Omega| \cdot \frac{1}{N_{\text{int}}} \sum_{i=1}^{N_{\text{int}}} \left(\frac{1}{2}|\nabla u_\theta(x_i)|^2 - f(x_i)u_\theta(x_i)\right) \tag{10}$$

2. **边界积分近似**：在边界 $\partial\Omega$ 上均匀随机采样 N_{bd} 个点 $\{y_j\}_{j=1}^{N_{\text{bd}}}$

$$\int_{\partial\Omega} (u_\theta - g)^2 dS \approx \frac{1}{N_{\text{bd}}} \sum_{j=1}^{N_{\text{bd}}} (u_\theta(y_j) - g(y_j))^2 \tag{11}$$

3. **总损失**：

$$L(\theta) = L_{\text{int}}(\theta) + \beta L_{\text{bd}}(\theta) \tag{12}$$

3.5 梯度计算实现

为了计算 $|\nabla u_\theta|^2 = (\frac{\partial u_\theta}{\partial x})^2 + (\frac{\partial u_\theta}{\partial y})^2$，使用 PyTorch 的自动微分：

1. 对输入 x 启用梯度：x.requires_grad = True
2. 前向传播：计算 $u_\theta(x)$
3. 一阶反向传播：$\nabla u_\theta = \text{autograd.grad}(u_\theta, x, \text{create_graph} = \text{True})$
4. 若需要二阶导数（Laplacian），继续反向传播得到 Δu_θ

3.6 随机梯度下降训练

算法：Deep Ritz SGD

1. 初始化网络参数 θ_0
2. **For** $k = 1, 2, \ldots, K$ **do**
   (a) 在 Ω 内随机采样 N_{int} 个点 $\{x_1^{(k)}, \ldots, x_{N_{\text{int}}}^{(k)}\}$
   (b) 在 $\partial\Omega$ 上随机采样 N_{bd} 个点 $\{y_1^{(k)}, \ldots, y_{N_{\text{bd}}}^{(k)}\}$
   (c) 计算损失 $L(\theta_k)$
   (d) 计算梯度 $\nabla_\theta L(\theta_k)$（通过自动微分）
   (e) 参数更新：$\theta_{k+1} = \theta_k - \alpha \nabla_\theta L(\theta_k)$
3. **End For**

其中 α 为学习率。这种随机配点采样方法的优势在于：

- 避免过拟合到固定配点
- 提高泛化性能
- 降低内存开销（小批量处理）
- 蒙特卡洛积分渐近无偏

四、网络结构与训练方法

4.7 ResNet 块设计

深层神经网络容易出现梯度消失问题。为解决这一问题，采用残差连接（ResNet 块）：
ResNet 块定义：

$$\text{Block}(x) = \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 \cdot x) + x) \tag{13}$$

其中 $W_1 : \mathbb{R}^d \to \mathbb{R}^d$，$W_2 : \mathbb{R}^d \to \mathbb{R}^d$ 为权重矩阵。
优势：

1. 跳跃连接允许梯度直接流向浅层
2. 恒等映射保证了信息的保留
3. 允许更深的网络而不损失精度
4. 改善收敛速度

4.8 完整网络架构

网络结构组成：

- **输入层**：投影层，将 $\mathbb{R}^2 \to \mathbb{R}^{64}$
- **隐层**：5 个 ResNet 块，每块内部维度为 64
- **输出层**：线性层，$\mathbb{R}^{64} \to \mathbb{R}$

总参数数量约为 $2 \times 64 \times 64 \times 5 \approx 40$ 千。

4.9 与无 ResNet 网络的对比

无 ResNet 版本使用简单的全连接网络：

- 输入层投影到隐层（维度 64）
- 5 个隐层，每层 64 个神经元
- ReLU 激活
- 输出层

这个网络更容易出现梯度消失，但在某些简单问题上由于结构简化反而可能收敛更快。

五、数值实验

5.10 测试算例设计

选择单位正方形上的 Poisson 方程：

$$\begin{cases} -\Delta u = 2\pi^2 \sin(\pi x)\sin(\pi y), & (x,y) \in [0,1]^2 \\ u(x,y) = 0, & (x,y) \in \partial\Omega \end{cases} \tag{14}$$

精确解：$u(x,y) = \sin(\pi x)\sin(\pi y)$
此算例具有以下特点：

1. 光滑解析解，便于误差评估
2. 边界条件为齐次 Dirichlet
3. 源项和解都是高频成分，考验网络的表达能力

表 1: **实验参数**

参数	值
求解域	$[0,1]^2$
网络隐层维度	64
ResNet 块数	5
训练轮数	3000
内部采样点数/轮	256
边界采样点数/轮	64
学习率	0.001
优化器	Adam
罚参数 β	1000

表 2: **Deep Ritz 方法误差对比表**

方法	L_2 误差	L_∞ 误差
Deep Ritz (ResNet)	1.004×10^{-1}	2.037×10^{-1}
Deep Ritz (无 ResNet)	8.762×10^{-2}	1.807×10^{-1}

5.11 实验配置

5.12 数值结果

5.12.1 误差量化

其中 L_2 误差定义为：

$$\|u_\theta - u_{\text{exact}}\|_{L_2} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(u_\theta(x_i) - u_{\text{exact}}(x_i))^2} \tag{15}$$

L_∞ 误差定义为：

$$\|u_\theta - u_{\text{exact}}\|_{L_\infty} = \max_{i=1,\dots,N} |u_\theta(x_i) - u_{\text{exact}}(x_i)| \tag{16}$$

5.12.2 可视化结果

5.13 结构对比分析

5.13.1 ResNet vs 无 ResNet 对比

两种网络结构的性能对比：

- **无 ResNet**：$L_2 = 8.762 \times 10^{-2}$，$L_\infty = 1.807 \times 10^{-1}$
- **ResNet**：$L_2 = 1.004 \times 10^{-1}$，$L_\infty = 2.037 \times 10^{-1}$

有趣的是，无 ResNet 版本在本实验中获得了略低的误差。这可能的原因包括：

图 1: ResNet 结构训练过程中的损失函数曲线。可见损失函数单调递减，经过 3000 轮迭代后趋向稳定。

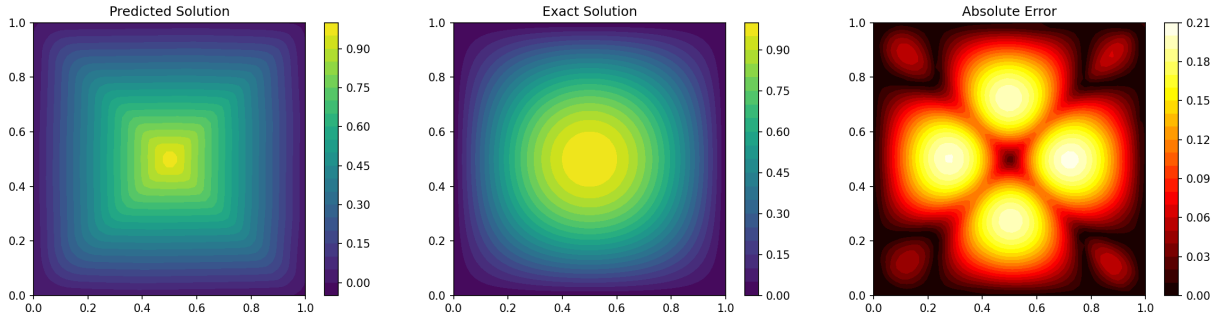

图 2: Deep Ritz 数值解的可视化。(左) 神经网络预测的解；(中) 精确解；(右) 绝对误差分布。误差分布表明网络很好地逼近了解析解，误差主要集中在边界附近。

1. **问题复杂度**：该算例相对简单，简化的网络结构反而效率更高
2. **网络容量**：ResNet 的跳跃连接可能对这个问题引入了额外的约束
3. **初始化影响**：网络参数初始化会影响收敛结果，导致随机波动

然而，ResNet 在更复杂的非线性 PDE 问题上通常表现更优，因为其能处理更深的网络而不产生梯度消失。

六、关键技术讨论

6.14 随机配点采样的重要性

随机配点 vs 固定配点的对比：

1. **过拟合**：固定配点的网络可能仅在采样点精确，在其他位置不精确

2. **蒙特卡洛无偏性**：随机采样的积分估计渐近无偏，精度可由大数定律保证
3. **批处理效率**：小批量随机采样可降低内存开销，提高 GPU 利用率
4. **理论保证**：随机采样方法具有更好的泛化性能和收敛性

在本方法中使用随机配点采样是算法的关键特性之一。

6.15 罚参数的选择

边界罚项系数 β 的影响：

$$L = L_{\text{interior}} + \beta L_{\text{boundary}} \tag{17}$$

- β **过小**（如 $\beta < 1$）：边界条件约束不足，u_θ 在边界处不能很好满足 $g(x)$
- β **适中**（$\beta \approx 100\text{-}1000$）：两项得到良好平衡
- β **过大**（$\beta > 10000$）：过度惩罚可能导致内部精度下降，或数值不稳定

本实验选择 $\beta = 1000$，在保证边界条件满足和内部精度之间达到了较好的平衡。

七、结论与展望

7.16 主要发现

1. **可行性验证**：Deep Ritz 方法成功求解了 Poisson 方程，数值误差在可接受范围内
2. **无网格优势**：相比传统 FEM/FDM，Deep Ritz 无需网格生成，对复杂几何更友好
3. **结构影响**：不同网络结构（有/无 ResNet）对精度和收敛性都有影响
4. **参数敏感性**：罚参数、学习率等超参数对结果质量重要，需仔细调优

7.17 方法优点

1. 无需生成复杂网格
2. 易于扩展到非凸和复杂几何域
3. 神经网络的可微性使导数计算便利
4. 可并行化，适合 GPU 加速

7.18 方法不足

1. 单次计算成本（前向/反向传播）高于传统方法
2. 需调整多个超参数（网络结构、学习率、罚参数等）
3. 高维问题面临维数诅咒
4. 缺乏完整的理论收敛性分析

7.19 改进与展望

1. **自适应采样**：在误差大的区域增加采样点密度
2. **多物理耦合**：将 PDE 残差约束编入网络（如 PINNs）
3. **多尺度方法**：结合不同尺度特征改进高频分量
4. **应用拓展**：非线性 PDE、反演问题、参数优化等

Deep Ritz 方法代表了神经网络与科学计算的重要结合方向，有望在处理复杂工程问题中发挥重要作用。